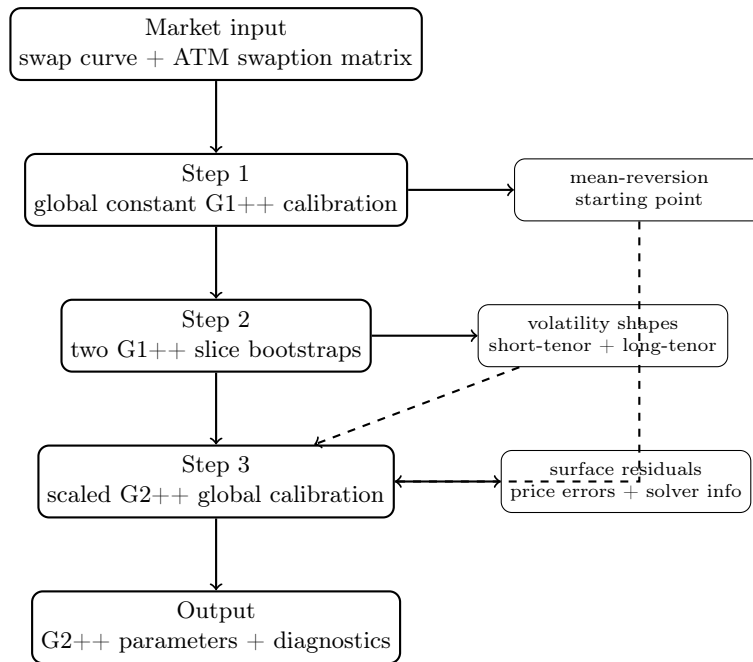


0.1. Bonino calibration architecture

The calibration module is organized as a staged pipeline. Each stage owns a specific responsibility: market-data preparation, one-factor initialization, two-factor reduced calibration, and diagnostics. The G2++ pricing kernel is used only after the volatility shapes have been prepared.



Module responsibilities

Module	Responsibility
<code>buildBoninoSurfacePackG2PP</code>	Normalize the swaption cube into expiry dates, swap tenors, maturities, volatility matrix, optional strikes, and optional weights.
<code>calibrateGlobalG1PP</code>	Calibrate a constant-parameter one-factor Gaussian model to the full swaption surface. This gives the mean-reversion starting point.
<code>bootstrapG1PPSlice</code>	Bootstrap one piecewise-constant G1++ volatility shape on a selected swaption slice. This function is used twice.
<code>initializeBoninoShapesG2PP</code>	Build the two G2++ volatility shapes from the short-tenor and long-tenor G1++ slice bootstraps.
<code>buildBoninoCalibrationSetG2PP</code>	Create all vanilla swaption instruments and their market target prices.
<code>boninoScaledSurfaceResidualG2PP</code>	Apply scaled volatility shapes to the G2++ model, price all instruments, and return the full residual vector.
<code>solveBoninoScaledG2PP</code>	Run the reduced global least-squares calibration.
<code>makeBoninoDiagnosticsG2PP</code>	Store calibrated parameters, residuals, model prices, market prices, and convergence information.

Implementation flow

The orchestrator should remain thin. Its role is to call each block in order and avoid embedding pricing or calibration details in the main script.

```

surfacePack = buildBoninoSurfacePackG2PP(...);

g1Global = calibrateGlobalG1PP(...);

sigmaShape = bootstrapG1PPSlice(..., "shortTenor", g1Global);
etaShape    = bootstrapG1PPSlice(..., "longTenor",  g1Global);

boninoState = initializeBoninoShapesG2PP( ...
    sigmaShape, etaShape);

boninoCalibSet = buildBoninoCalibrationSetG2PP( ...
    surfacePack, pricers);
  
```

```
[thetaStar, solverInfo] = solveBoninoScaledG2PP( ...
    boninoCalibSet, boninoState);

diagnostics = makeBoninoDiagnosticsG2PP( ...
    thetaStar, solverInfo, boninoCalibSet, boninoState);
```

Control map

Stage	Input	Output
Market normalization	Swaption cube / surface	surfacePack
Global G1++ calibration	Full surface	g1Global
Short-tenor bootstrap	Shortest-tenor slice	sigmaShape
Long-tenor bootstrap	Longest-tenor slice	etaShape
Scaled G2++ calibration	Full surface + two shapes	G2++ calibrated parameters
Diagnostics	Residuals + prices	Calibration report

Design rule

The G2++ model class should not know how market data are stored. The calibration module converts market data into simple calibration structures, then calls the model only through pricing and volatility-state methods.

market data → calibration pack → model residual → solver → diagnostics
--